



RUNTS-CAI

Documentazione tecnica del sistema

*Versione 1.0 — Draft
23 maggio 2026*

Indice

1. Panoramica del sistema

2. Architettura

3. Scraper RUNTS-CAI

4. Database (SQLite)

5. Geocoder (Nominatim)

6. Web app (FastAPI)

7. Deploy ed esecuzione

Documento generato a partire dai file Markdown in *docs/*. Le sezioni che seguono riportano la fotografia dello stato attuale del sistema RUNTS-CAI e costituiscono materiale di input per le specifiche OpenSpec.

1. Panoramica del sistema

Fotografia dello stato attuale del sistema. Questo documento è materiale di input per la generazione di specifiche OpenSpec.

Scopo del progetto

Il sistema **RUNTS-CAI** acquisisce, normalizza, persiste e pubblica i dati delle sezioni del **Club Alpino Italiano** iscritte al **Registro Unico Nazionale del Terzo Settore (RUNTS)** del Ministero del Lavoro.

L'obiettivo è disporre di un dataset locale interrogabile e di una mini-applicazione web di consultazione che mostri la lista, il dettaglio e la mappa delle sezioni CAI registrate.

Attori e responsabilità

Attore	Responsabilità
Operatore (CLI)	Lancia manualmente lo scraper e il geocoder per popolare e arricchire il DB.
Scraper (<code>scraper/</code>)	Automatizza la ricerca su <code>servizi.lavoro.gov.it/runts</code> ed estrae i dati di dettaglio di ciascun ente.
Geocoder (<code>scraper/geocoder.py</code>)	Aggiunge coordinate lat/lon agli enti interrogando Nominatim (OpenStreetMap).
Database (<code>runts.db</code> , SQLite)	Persiste gli enti in un'unica tabella <code>enti</code> con upsert per <code>id_runts</code> .
Web app (<code>web/</code> , FastAPI)	Consulta in sola lettura <code>runts.db</code> e serve lista filtrabile, dettaglio e mappa Leaflet.
Utente finale (browser)	Naviga la lista, applica filtri, apre il dettaglio del singolo ente, visualizza la mappa.

Componenti principali

Il sistema è composto da tre macro-componenti, debolmente accoppiati e condivisi solo attraverso il file `runts.db` :

1. **Scraper RUNTS** — Python + Playwright async, esegue una ricerca filtrata per la denominazione "CLUB ALPINO ITALIANO", pagina i risultati, naviga al dettaglio di ogni ente, estrae i campi e li salva nel DB.
2. **Geocoder Nominatim** — script Python standalone che legge dal DB gli enti senza coordinate e popola `lat` / `lon` via `nominatim.openstreetmap.org` , rispettando il rate

MONTAGNA SERVIZI S.C.P.A.

Via Errico Petrella 19 - 20124 - Milano (MI)

<https://montagnaservizi.com>

PEC: montagnaserviziscpa@legalmail.it - email: info@montagnaservizi.com

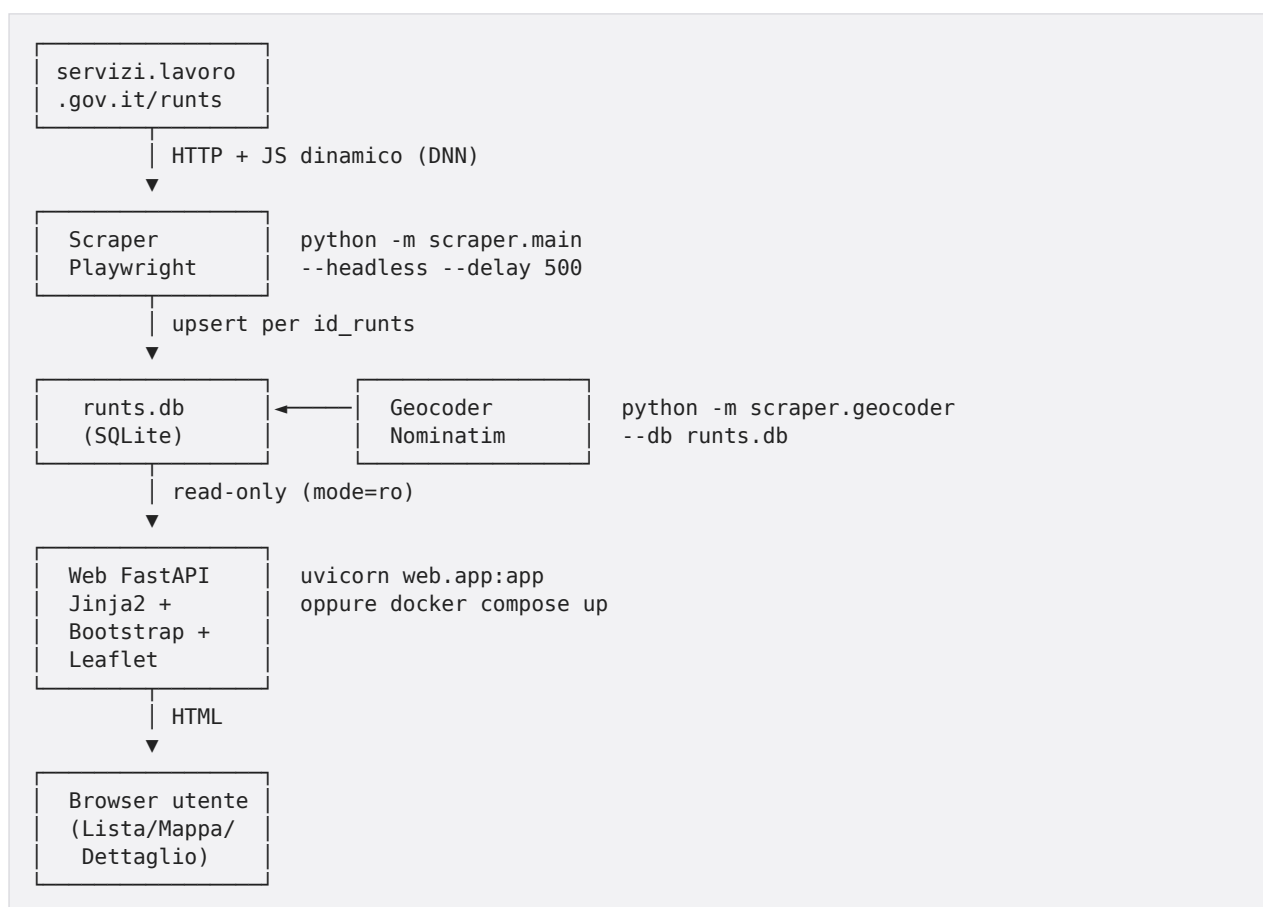
C.F./P.IVA: 11790660960

limit di 1 req/s.

3. **Web app FastAPI** — applicazione Jinja2/Bootstrap che apre `runts.db` in sola lettura e serve due route HTML (`/` , `/ente/<id_runts>`), con vista lista, vista mappa e dettaglio singolo ente.

Lo scraper e il web sono **isolati**: hanno requirements separati, vivono in directory separate (`scraper/` , `web/`) e si parlano esclusivamente via il file `runts.db` alla radice del progetto.

Flusso end-to-end



Stack tecnologico

Strato	Tecnologia
Linguaggio	Python 3.12
Scraping	Playwright (Chromium async)
HTTP client (geocoder)	<code>urllib.request</code> (no dipendenze esterne)
Storage	SQLite con WAL journal

Strato	Tecnologia
Backend web	FastAPI + Uvicorn
Templating	Jinja2
UI	Bootstrap 5.3 (CDN)
Mappe	Leaflet 1.9.4 + tile OpenStreetMap (CDN)
Geocoding	Nominatim OSM (<code>nominatim.openstreetmap.org</code>)
Test	pytest
Deploy web	Docker + docker-compose

Vincoli e principi guida

- **Costo zero per servizi terzi:** nessuna API key richiesta (Nominatim e OSM sono gratuiti, Leaflet via CDN).
- **Idempotenza:** rieseguire scraper o geocoder non duplica record né riscrive coordinate già presenti se non necessario.
- **Separazione dei concern:** scraper e web non si importano a vicenda; il DB è l'unica interfaccia.
- **Read-only sul DB lato web:** il container web monta `runts.db` come `:ro` ; nessuna possibilità di scrittura dal lato HTTP.
- **Resilienza dello scraper:** errori sul singolo ente non interrompono il batch; viene loggato e si prosegue.

Glossario

Termine	Significato
RUNTS	Registro Unico Nazionale del Terzo Settore, gestito dal Ministero del Lavoro.
Ente	Singola organizzazione iscritta al RUNTS (nel nostro dominio: una sezione CAI).
Sezione del registro	Categoria di iscrizione al RUNTS (es. APS, ODV, altri enti del terzo settore).
<code>id_runts</code>	Identificatore univoco assegnato dal RUNTS a un ente (numero di repertorio).
Codice fiscale	CF dell'ente, usato come chiave secondaria (<code>UNIQUE</code>).

Termine	Significato
Sede legale	Indirizzo registrato dell'ente; dal RUNTS si estraggono stato, indirizzo, civico, comune, provincia, regione, CAP.
Geocodifica	Conversione comune/provincia in coordinate geografiche (lat/lon) via Nominatim.
Upsert	INSERT OR REPLACE su <code>id_runts</code> come chiave primaria.
DNN	DotNetNuke, il CMS su cui gira il portale RUNTS (rilevante perché i campi vengono caricati via JavaScript dopo <code>networkidle</code>).
Nominatim	Servizio di geocoding gratuito basato su dati OpenStreetMap.

Dataset di riferimento

Al momento della scrittura, l'ente `id_runts=83894` (CAI Pisa) è usato come caso di test per validare estrazione di sede legale completa e visualizzazione mappa nel dettaglio. Il dataset complessivo è dell'ordine di ~226 enti (Sezioni del CAI nel registro).

2. Architettura

Fotografia dei componenti, delle loro interfacce e dei loro confini.

Struttura del repository

```

RUNTS/
├── Dockerfile                # immagine container per la sola web app
├── docker-compose.yml        # servizio "web" con bind di runts.db in :ro
├── runts.db                  # database SQLite (popolato da scraper + geocoder)
├── openspec/                 # spec OpenSpec (gestita da ClaudeCode)
│   ├── specs/                # capability con spec correnti
│   └── changes/              # change proposte e archiviate
├── docs/                     # materiale per generare nuove spec (questo dir)
├── scraper/                  # package Python dello scraper + geocoder
│   ├── __init__.py
│   ├── main.py               # entrypoint CLI: python -m scraper.main
│   ├── scraper.py            # logica Playwright: ricerca + estrazione
│   ├── db.py                 # schema, migrazioni, upsert
│   ├── geocoder.py           # script Nominatim, eseguibile come modulo
│   ├── test_sede_legale.py    # test pytest sulla regex/estrazione sede
│   └── requirements.txt       # playwright + pytest
├── web/                      # web app FastAPI
│   ├── app.py                # routes / e /ente/<id_runts>
│   ├── requirements.txt       # fastapi, uvicorn[standard], jinja2
│   ├── static/               # cartella montata (attualmente vuota)
│   └── templates/            # Jinja2
│       ├── base.html         # layout Bootstrap + navbar
│       ├── list.html         # lista + filtri + toggle Lista/Mappa
│       ├── detail.html       # scheda ente + mappa Leaflet condizionale
│       └── 404.html          # ente non trovato

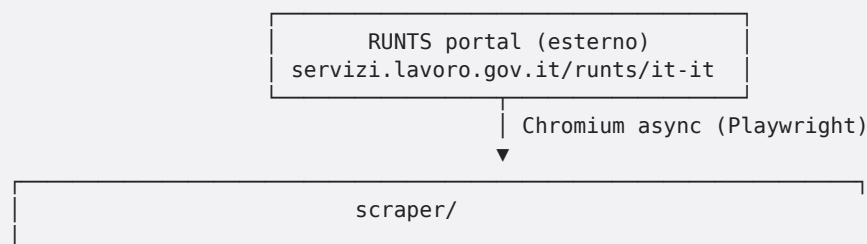
```

Vincoli di layout

Lo scraper è isolato in `scraper/` come package importabile (`import scraper` funziona). I file dello scraper non devono comparire alla radice del progetto. Le dipendenze sono in `scraper/requirements.txt` per lo scraper e `web/requirements.txt` per la web app — i due file non condividono pacchetti.

Il file `runts.db` vive **alla radice del progetto** (non dentro `scraper/`): è l'unico canale di comunicazione tra scraper, geocoder e web app.

Diagramma logico dei componenti



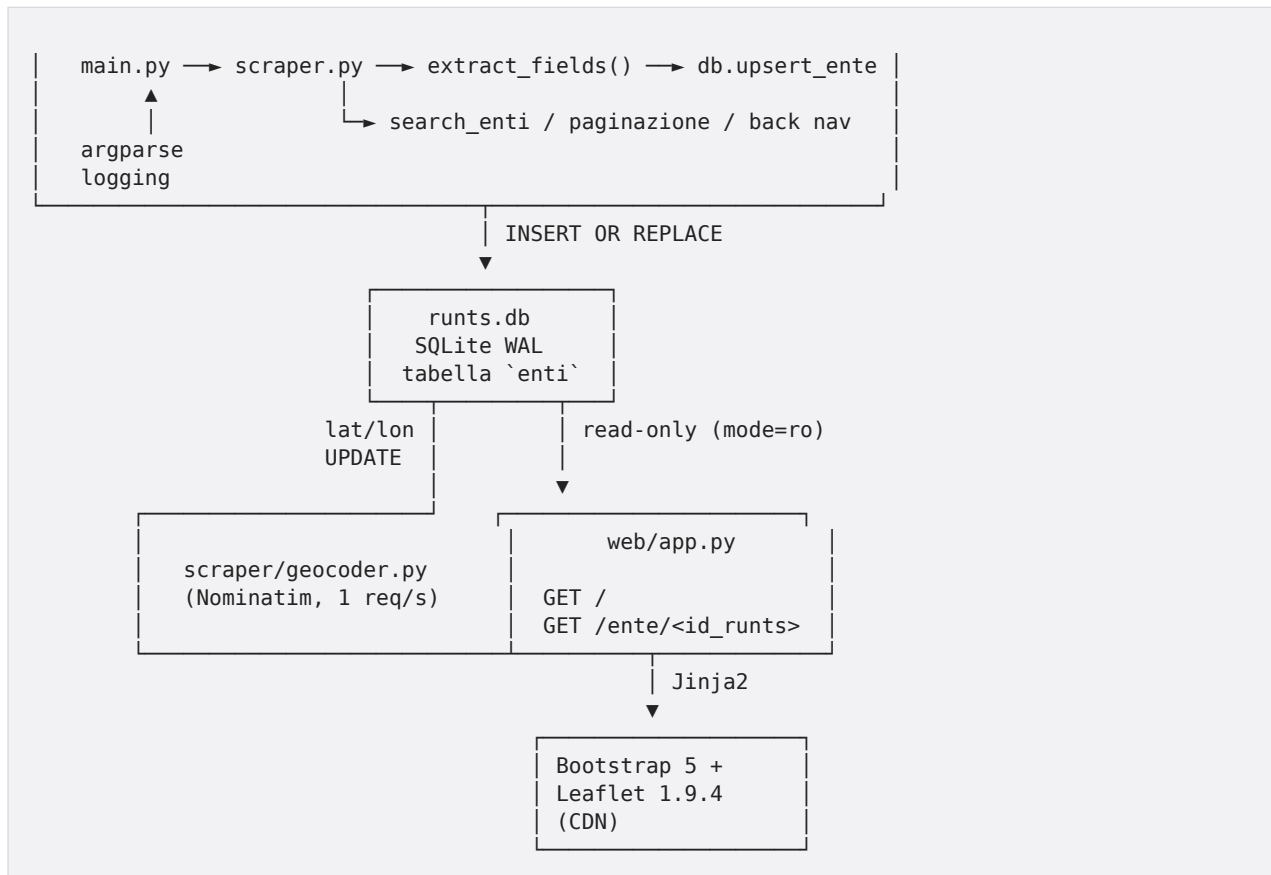
MONTAGNA SERVIZI S.C.P.A.

Via Errico Petrella 19 - 20124 - Milano (MI)

<https://montagnaservizi.com>

PEC: montagnaserviziscpa@legalmail.it - email: info@montagnaservizi.com

C.F./P.IVA: 11790660960



Interfacce tra componenti

Scraper → DB

- API interna Python: `db.init_db(path) -> sqlite3.Connection` e `db.upsert_ente(conn, data) -> "inserted" | "updated"`.
- Chiave primaria: `id_runts` (fallback `codice_fiscale`).
- Tutti i campi sono `TEXT` tranne `lat`, `lon` (`REAL`) e `updated_at` (`TEXT NOT NULL` con timestamp ISO UTC).

Geocoder → DB

- Apertura diretta del DB (`sqlite3.connect`), `row_factory = sqlite3.Row`.
- Lettura: `SELECT id_runts, sede_comune, sede_regione FROM enti WHERE lat IS NULL OR lon IS NULL`.
- Scrittura: `UPDATE enti SET lat = ?, lon = ? WHERE id_runts = ?`.

Geocoder → Nominatim

- Endpoint: `https://nominatim.openstreetmap.org/search`.

- User-Agent obbligatorio (per Nominatim policy): `runts-cai-geocoder/1.0` (<https://github.com/piccioli/runts>) .
- Parametri: `q=<Comune>, <Regione>, Italia , format=json , limit=1` .
- Rate limit applicato: `time.sleep(1)` dopo ogni richiesta.

Web → DB

- `sqlite3.connect("file:<DB_PATH>?mode=ro", uri=True, check_same_thread=False)` .
- `row_factory = sqlite3.Row` per accesso per nome di colonna.
- Una connessione per request (apertura + `try/finally` di `close`) .

Web → Browser

- Risposta `text/html` via `TemplateResponse` di FastAPI.
- Asset frontend (Bootstrap, Leaflet, tile OSM) caricati da CDN, **nessun bundle locale**.
- La cartella `web/static/` è montata su `/static` ma attualmente è vuota.

Configurazione

Variabile	Default	Componente	Note
<code>DB_PATH</code>	<code>/app/runts.db</code>	web app	Letta da <code>os.environ</code> in <code>web/app.py</code> .
<code>--db</code>	<code>runts.db</code>	scraper / geocoder	Argomento CLI.
<code>--headless</code>	<code>True</code>	scraper	<code>--no-headless</code> per debug visivo del browser.
<code>--delay</code>	<code>500 ms</code>	scraper	Pausa tra una pagina di dettaglio e la successiva.
<code>--verbose</code> / <code>-v</code>	<code>False</code>	scraper / geocoder	Abilita log a livello DEBUG.

Non esistono altri file `.env` , secret manager o config esterni.

Modello di esecuzione

Il sistema **non ha scheduler interno**. Ogni operazione è on-demand:

1. **Aggiornamento dati:** l'operatore lancia `python -m scraper.main` quando serve rinfrescare il DB. La durata dipende dal numero di enti ($\sim 226 \times \sim 3$ s di rendering + delay 0.5 s $\approx$ 12-15 minuti).

2. **Geocodifica:** l'operatore lancia `python -m scraper.geocoder --db runts.db` dopo lo scraper. Durata ~4 minuti per 226 enti (rate limit 1 req/s).
3. **Servizio web:** avviato una tantum con `docker compose up -d`, resta su finché non viene fermato; legge il DB ad ogni richiesta HTTP (nessuna cache applicativa).

Lo scraper può essere rieseguito mentre il container web è attivo: SQLite in WAL mode + montaggio `:ro` lato web permette letture concorrenti senza riavvio.

Confini e non-goal architetturali

- Nessuna API REST/JSON: la web app serve solo HTML.
- Nessuna autenticazione: l'app è pensata per consultazione pubblica/interna.
- Nessuna scrittura via HTTP: il DB è read-only dal lato applicativo.
- Nessun job scheduler, message queue, cache distribuita.
- Nessun bundler frontend (no webpack/vite); tutto via CDN.
- Nessun ORM: SQL diretto via `sqlite3` standard library.

3. Scraper RUNTS-CAI

Comportamento osservato del modulo `scraper/` allo stato attuale.

Scopo

Lo scraper acquisisce in modo automatico la lista delle sezioni del CAI dal portale RUNTS, estraendo da ciascuna scheda di dettaglio i campi strutturati, e li persiste nel database SQLite locale.

Entrypoint CLI

File: `scraper/main.py` . Eseguitibile come modulo:

```
python -m scraper.main [--db PATH] [--headless | --no-headless] [--delay MS] [--verbose]
```

Argomenti:

Argomento	Default	Significato
<code>--db PATH</code>	<code>runts.db</code>	Percorso del file SQLite (verrà creato se non esiste).
<code>--headless / --no-headless</code>	<code>True</code>	Modalità Chromium. <code>--no-headless</code> apre una finestra visibile per debug.
<code>--delay MS</code>	<code>500</code>	Pausa in millisecondi dopo aver estratto un dettaglio, prima di tornare alla lista.
<code>--verbose , -v</code>	<code>off</code>	Livello log <code>DEBUG</code> .

A fine esecuzione viene stampato un **report finale** con il conteggio di enti processati, inseriti, aggiornati, errori di salvataggio e totale presente nel DB (vedi `main.py:88-98`).

URL e selettori target

Costante	Valore	Funzione
<code>SEARCH_URL</code>	<code>https://servizi.lavoro.gov.it/runts/it-it/Ricerca-enti</code>	Form di ricerca pubblico RUNTS.
<code>DETAIL_URL_PATTERN</code>	<code>**/Ricerca-enti/Ente*</code>	Pattern per <code>wait_for_url</code> dopo click su "Dettaglio".

Il portale gira su DotNetNuke e popola i campi via JavaScript: lo scraper attende sempre `networkidle` e poi i selettori specifici prima di estrarre.

Flusso di esecuzione

1. `init_db(args.db)` — apre la connessione SQLite, abilita WAL, applica schema + migrazioni.
2. `run_scraper(denominazione="CLUB ALPINO ITALIANO", headless, delay_ms)` — orchestra l'intero scraping; restituisce `list[dict]` pronti per upsert.
3. **Loop di upsert** — per ogni dict restituito, `upsert_ente(conn, entity)` ritorna `"inserted"` o `"updated"` e i contatori vengono aggiornati.
4. **Report finale** — `get_stats(conn)` legge il totale aggiornato e viene stampato il riepilogo.

In caso di **eccezione fatale durante** `run_scraper`, la connessione DB viene chiusa e il processo esce con `sys.exit(1)` (`main.py:67-70`).

Fase 1 — Ricerca (`search_enti`)

1. Naviga su `SEARCH_URL` con `wait_until="domcontentloaded"`.
2. Attesa fissa di 2000 ms per il rendering DNN.
3. Riempie `input[id*="denominazione" i]` con la denominazione (default `"CLUB ALPINO ITALIANO"`).
4. Click su `input[type="submit"]`.
5. Attende `input[value="Dettaglio"]` (timeout 30 s) come segnale di tabella risultati popolata.
6. Pausa di 500 ms per stabilizzare la pagina.

Conteggio risultati

- **Totale enti** (`_get_total_items`): legge l'attributo `value` di `[id*="hdnListEntiTotalItems"]`. Fallback a 0 se assente.
- **Paginazione** (`_get_page_info`): estrae (N, M) dalla label `[id*="spnLabelNumeroPagina"]` con la regex `(\d+)\s+di\s+(\d+)`. Fallback a (1, 1).

Se `total_items == 0` lo scraper logga `"Nessun ente trovato per '<denominazione>'."` e restituisce lista vuota senza errore.

Fase 2 — Paginazione (`_go_to_next_page`)

1. Controlla (`cur, tot`) correnti; se `cur >= tot` ritorna `False`.
2. Click su `a[id*="ltlProssimaPagina"]`.

3. **Attesa specifica** della label paginazione che mostra "Pagina <expected_page>" (timeout 20 s) — workaround per la race condition tra DOM aggiornato e label DNN.
4. Attesa di `input[value="Dettaglio"]` (timeout 10 s).

Se la transizione fallisce, viene loggato un warning e si ritorna `False` (il ciclo principale si fermerà).

Fase 3 — Estrazione metadati di riga (`_collect_row_metadata`)

Per ogni `tr` in `table tbody`, legge le prime tre `td`:

- `denominazione` (cella 0)
- `sede_comune` (cella 1)
- `sezione_registro` (cella 2)

Queste informazioni sono raccolte **prima** di navigare al dettaglio: vengono unite ai campi estratti dalla scheda di dettaglio per gli eventuali valori non trovati lì.

Fase 4 — Navigazione al dettaglio

Per ogni indice `i` dei pulsanti `input[value="Dettaglio"]`:

1. Click sull'*i*-esimo pulsante.
2. `page.wait_for_url(DETAIL_URL_PATTERN, timeout=15s)`.
3. `page.wait_for_load_state("networkidle", timeout=20s)`.
4. `extract_fields(page)` — vedi fase 5.
5. `await asyncio.sleep(delay_ms / 1000)`.
6. `_back_to_results(page)` — `page.go_back()` + attesa pulsanti Dettaglio (timeout 10 s).
7. Se il back fallisce, lo scraper **ri-esegue la ricerca** e naviga fino alla pagina corrente con `_go_to_next_page` ripetuto.

In caso di eccezione su un singolo ente, viene loggato l'errore con denominazione e indice, e si tenta il back; se il back fallisce si ri-esegue la ricerca dalla pagina 1.

I dialog del browser sono accettati automaticamente: `page.on("dialog", lambda d: asyncio.ensure_future(d.accept()))`.

Fase 5 — Estrazione campi (`extract_fields`)

5.1 Campi base via selettori ID

Chiave	Selettore (substring match)	Note
id_runts	spnRepertorio	Numero di repertorio RUNTS. Usato come chiave primaria.
codice_fiscale	spnCodiceFiscale	Codice fiscale dell'ente.
pec	spnEmailPec	Email PEC.
sito_web	spnSitoInternet	URL sito web.

Attesa preliminare di `[id*="spnRepertorio"]` (timeout 10 s) come gate sull'avvenuto rendering del dettaglio.

5.2 Sede legale via JS scoped

Lo script `_EXTRACT_SEDE_LEGALE_JS` cerca un container `[id*="divSedeLegale"]` e all'interno estrae i campi via selettori con suffisso `SL` (Sede Legale):

Chiave nel dict	Selettore	Colonna DB
stato	<code>[id*="spnStatoSL"]</code>	sede_stato
provincia	<code>[id*="spnProvinciaSL"]</code>	sede_provincia
comune	<code>[id*="spnComuneSL"]</code>	sede_comune
indirizzo	<code>[id*="spnIndirizzoSL"]</code>	sede_indirizzo
civico	<code>[id*="spnCivicoSL"]</code>	sede_civico
cap	<code>[id*="spnCAP_SL"]</code>	sede_cap
regione	<code>[id*="spnRegioneSL"]</code>	sede_regione

Il container `divSedeLegale` **scopa** l'estrazione, evitando collisioni con altri blocchi (es. sede operativa, sede amministrativa).

5.2.1 Derivazione regione da sigla provincia

Se `sede_regione` non viene trovata nel DOM ma `sede_provincia` sì, viene applicata la mappa statica `_PROVINCIA_TO_REGIONE` (sigle di tutte le province italiane). Se la sigla non è mappata, viene loggato un debug e la regione resta nulla.

Esempi: `"PI"` → `"Toscana"`, `"BZ"` → `"Trentino-Alto Adige"`, `"MI"` → `"Lombardia"`.

5.3 Data di iscrizione

`[id*="spnIscrittoIl"]` con strip del prefisso `"Iscritto il "` (case insensitive) → `data_iscrizione`.

5.4 Coppie label/valore generiche

Lo script `_EXTRACT_BOLD_JS` itera su `.ente_bold`, prende l'elemento adiacente `[class*="ente_testo"]:not(.ente_bold)` e restituisce un dict `{label: value}`. Le label vengono normalizzate da `_normalize_label`:

Label italiana (sostringa, lowercase)	Colonna DB
sezione del registro / sezione	sezione_registro
forma giuridica	forma_giuridica
natura giuridica	natura_giuridica
codice fiscale	codice_fiscale
email pec	pec
sito internet	sito_web
denominazione	denominazione
provincia	sede_provincia
comune	sede_comune
regione	sede_regione
indirizzo	sede_indirizzo
cap	sede_cap

Label sconosciute con lunghezza ≤ 50 caratteri vengono incluse con chiave `snake_case` derivata; quelle più lunghe (header di sezione) sono scartate. Una chiave già presente nel dict non viene sovrascritta.

5.5 Rappresentante legale e raw_json

- Tutto il testo del `<body>` viene letto e troncato ai primi 10000 caratteri in `data["raw_json"]` come backup grezzo per debug/verifiche.
- Regex sul body: `Rappresentante legale\s+S[\i]\b.*?Nome\s+(\S+). *?Cognome\s+(\S+)` (case insensitive, dotall). Se trovata, popola `rappresentante_legale` come "Cognome Nome" (solo se non già presente).

5.6 URL del dettaglio

`data["url_dettaglio"] = page.url` viene sempre impostato (URL diretto della scheda RUNTS).

Configurazione browser

```
browser = pw.chromium.launch(headless=headless)
context = browser.new_context(
    user_agent=(
        "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) "
        "AppleWebKit/537.36 (KHTML, like Gecko) "
```

MONTAGNA SERVIZI S.C.P.A.

Via Errico Petrella 19 - 20124 - Milano (MI)

<https://montagnaservizi.com>

PEC: montagnaserviziscpa@legalmail.it - email: info@montagnaservizi.com

C.F./P.IVA: 11790660960

```
)  
    "Chrome/124.0.0.0 Safari/537.36"  
)
```

User-agent mascherato come Chrome su macOS per ridurre il rischio di blocchi lato server.
Nessuna persistenza di stato/cookies tra esecuzioni.

Logging

Livello	Eventi
INFO	navigazione, totale enti, cambio pagina, ogni ente processato ([N/TOT] <denominazione>), apertura DB, report finale.
WARNING	timeout label paginazione, errore Nominatim (geocoder), errore salvataggio, back navigation fallita.
ERROR	eccezione su singolo ente, errore fatale globale.
DEBUG	dettagli di estrazione JS, mismatch provincia→regione, fallback go_back.

Formato: `%(asctime)s %(levelname)-8s %(message)s` con `datefmt="%H:%M:%S"` .

Test esistenti

`scraper/test_sede_legale.py` (pytest) verifica l'estrazione della sede legale per casi noti.
Va eseguito nell'ambiente con `playwright` installato.

Comportamenti non garantiti

- Estrazione dipende dalla struttura HTML/ID del portale RUNTS; cambi lato server possono rompere lo scraping.
- Nessun retry automatico in caso di errore di rete: l'ente fallito viene saltato e segnalato come errore.
- Lo scraper non gestisce captcha o blocchi lato server diversi dall'errore generico.
- Le label normalizzate da `_normalize_label` con la fallback `snake_case` possono produrre colonne extra non presenti nello schema; queste vengono ignorate da `upsert_ente` perché filtrate sulla lista `columns` (vedi spec database).

4. Database (SQLite)

Schema, migrazioni e regole di persistenza implementati in `scraper/db.py`.

File e modalità

- **Percorso default:** `runs.db` alla radice del progetto (configurabile via `--db` per `scraper/geocoder` e via `DB_PATH` per la web app).
- **Engine:** SQLite locale, accesso via `sqlite3` standard library.
- **Journal mode:** `WAL` abilitato in `init_db` (`PRAGMA journal_mode=WAL`). Consente letture concorrenti da parte della web app mentre lo scraper scrive.
- **Row factory:** `sqlite3.Row` ovunque, per accesso per nome di colonna.
- **Read-only lato web:** la connessione lato `web/app.py` usa l'URI `file:<path>?mode=ro` con `uri=True`, garantendo che il container non possa modificare il DB.

Schema della tabella `enti`

Definizione canonica in `scraper/db.py` (costante `SCHEMA`):

```
CREATE TABLE IF NOT EXISTS enti (  
    id_runs          TEXT PRIMARY KEY,  
    codice_fiscale   TEXT UNIQUE,  
    denominazione    TEXT,  
    forma_giuridica  TEXT,  
    natura_giuridica TEXT,  
    sede_stato       TEXT,  
    sede_indirizzo   TEXT,  
    sede_civico      TEXT,  
    sede_comune      TEXT,  
    sede_provincia   TEXT,  
    sede_regione     TEXT,  
    sede_cap         TEXT,  
    lat              REAL,  
    lon              REAL,  
    data_iscrizione  TEXT,  
    sezione_registro TEXT,  
    settori_attivita TEXT,  
    rappresentante_legale TEXT,  
    sito_web         TEXT,  
    pec              TEXT,  
    url_dettaglio    TEXT,  
    raw_json         TEXT,  
    updated_at       TEXT NOT NULL  
);
```

Vincoli

Colonna	Vincolo	Significato
id_runts	PRIMARY KEY	Identificatore RUNTS, obbligatorio per ogni record persistito.
codice_fiscale	UNIQUE	Non possono esistere due enti con lo stesso CF.
updated_at	NOT NULL	Timestamp ISO UTC dell'ultimo upsert.

Tutte le altre colonne sono nullable: i campi non trovati nella scheda di dettaglio vengono salvati come `NULL`.

Significato delle colonne

Colonna	Origine dati	Esempio
id_runts	spnRepertorio (RUNTS)	83894
codice_fiscale	spnCodiceFiscale o normalizzazione label	93000990503
denominazione	tabella risultati o label	CLUB ALPINO ITALIANO SEZIONE DI PISA
forma_giuridica	label "Forma giuridica"	Associazione
natura_giuridica	label "Natura giuridica"	Associazione non riconosciuta
sede_stato	spnStatoSL	ITALIA
sede_indirizzo	spnIndirizzoSL	Via Cesare Battisti
sede_civico	spnCivicoSL	1
sede_comune	spnComuneSL	PISA
sede_provincia	spnProvinciaSL	PI
sede_regione	spnRegioneSL o derivata da provincia	Toscana
sede_cap	spnCAP_SL	56125
lat , lon	geocoder (Nominatim)	43.7159, 10.4018
data_iscrizione	spnIscrittoIl (con prefisso "Iscritto il" rimosso)	15/02/2023
sezione_registro	tabella risultati o label	Associazioni di promozione sociale
settori_attivita	label generica (se presente)	testo libero

MONTAGNA SERVIZI S.C.P.A.

Via Errico Petrella 19 - 20124 - Milano (MI)

<https://montagnaservizi.com>

PEC: montagnaserviziscpa@legalmail.it - email: info@montagnaservizi.com

C.F./P.IVA: 11790660960

Colonna	Origine dati	Esempio
representante_legale	regex sul body (formato Cognome Nome)	ROSSI MARIO
sito_web	spnSitoInternet	https://...
pec	spnEmailPec	cai.pisa@pec.it
url_dettaglio	URL della scheda RUNTS	https://servizi.lavoro.gov.it/...
raw_json	primi 10000 caratteri del body	testo grezzo per audit
updated_at	datetime.now(timezone.utc).isoformat()	2026-05-22T18:45:12.123456+00:00

Migrazioni

`init_db()` esegue, dopo `CREATE TABLE IF NOT EXISTS`, un ciclo di `ALTER TABLE` definiti in `_MIGRATIONS`. Ogni `ALTER` è racchiuso in `try/except sqlite3.OperationalError` per essere idempotente (se la colonna esiste già, errore silenzioso).

`_MIGRATIONS` attualmente contiene:

```
"ALTER TABLE enti ADD COLUMN sede_stato TEXT",
"ALTER TABLE enti ADD COLUMN sede_civico TEXT",
"ALTER TABLE enti ADD COLUMN lat REAL",
"ALTER TABLE enti ADD COLUMN lon REAL",
```

Regole per nuove migrazioni

1. Aggiungere la colonna a `SCHEMA` (per le installazioni nuove).
2. Aggiungere l' `ALTER TABLE ... ADD COLUMN ...` a `_MIGRATIONS` (per le installazioni esistenti).
3. Aggiungere la colonna alla lista `columns` di `upsert_ente()` (altrimenti il valore non verrà mai scritto).

Non c'è un sistema di versionamento esplicito (no `PRAGMA user_version`): l'idempotenza è garantita solo dal `try/except`.

Upsert (`upsert_ente`)

Firma: `upsert_ente(conn: sqlite3.Connection, data: dict) -> "inserted" | "updated"`.

Comportamento:

1. Si fa una copia del dict in input e si imposta `data["updated_at"] = datetime.now(timezone.utc).isoformat()`.

2. Se `raw_json` non è presente, viene popolato con `json.dumps(data, ensure_ascii=False)`.
3. Si calcola `id_runts = data.get("id_runts") or data.get("codice_fiscale")`. Se entrambi sono nulli, viene sollevato `ValueError("Record senza id_runts né codice_fiscale, impossibile fare upsert")`.
4. Si verifica esistenza con `SELECT 1 FROM enti WHERE id_runts = ?` per decidere il valore di ritorno (`"inserted"` o `"updated"`).
5. Si esegue `INSERT OR REPLACE INTO enti (...) VALUES (...)` su una lista fissa di 23 colonne.
6. `conn.commit()` chiude la transazione.

Lista canonica di colonne scritte dall'upsert

```
id_runts, codice_fiscale, denominazione, forma_giuridica,
natura_giuridica, sede_stato, sede_indirizzo, sede_civico,
sede_comune, sede_provincia, sede_regioni, sede_cap,
lat, lon, data_iscrizione, sezione_registro, settori_attivita,
rappresentante_legale, sito_web, pec,
url_dettaglio, raw_json, updated_at
```

Eventuali chiavi extra nel dict (es. fallback snake_case da `_normalize_label`) vengono **ignorate**: non c'è schema flessibile.

Conseguenze di `INSERT OR REPLACE`

Un upsert **riscrive sempre tutte le colonne**. Se in una nuova esecuzione un campo è assente, viene sovrascritto con `NULL`. Questo significa che:

- Lo scraper deve fornire i campi più completi possibili a ogni run.
- Il geocoder, che agisce con `UPDATE` mirato su `lat / lon`, non viene invalidato da un'esecuzione successiva dello scraper a patto che l' `upsert_ente` riceva `lat / lon` valorizzati nel dict — ma lo scraper **non** li popola, quindi un rerun dello scraper **azzera le coordinate**.

⚠ Questo è un comportamento attuale del sistema da tenere presente nelle specifiche: dopo un rerun completo dello scraper occorre rilanciare il geocoder.

Lettura aggregata (`get_stats`)

`get_stats(conn) -> {"total": int}` : ritorna il numero di righe nella tabella `enti`. Usato solo nel report finale dello scraper. Non c'è caching.

Indici

Non esistono indici espliciti oltre alla `PRIMARY KEY` su `id_runts` e all' `UNIQUE` su `codice_fiscale`. Per i volumi attuali (~226 righe) la web app esegue scan completi senza

MONTAGNA SERVIZI S.C.P.A.

Via Errico Petrella 19 - 20124 - Milano (MI)

<https://montagnaservizi.com>

PEC: montagnaserviziscpa@legalmail.it - email: info@montagnaservizi.com

C.F./P.IVA: 11790660960

problemi di performance. Eventuali indici aggiuntivi (es. su `sede_regione` , `sezione_registro`) sarebbero utili solo a partire da decine di migliaia di righe.

Sicurezza e integrità

- **Nessun input utente arriva al DB:** lo scraper scrive dati pubblici e la web app è read-only.
- Le query della web app usano sempre **parametri posizionali** (`?`) con la `execute` , mai concatenazione di stringhe: protezione completa da SQL injection.
- Non esiste backup automatico né rotation; il DB è committato/gestito a mano dall'operatore.

5. Geocoder (Nominatim)

Comportamento osservato di `scraper/geocoder.py`.

Scopo

Arricchire gli enti già presenti nel DB con coordinate geografiche (lat, lon) da usare per la mappa Leaflet della web app. Il geocoder è **uno script standalone**, eseguibile separatamente dallo scraper.

Entrypoint CLI

File: `scraper/geocoder.py`. Eseguibile come modulo:

```
python -m scraper.geocoder [--db PATH] [--verbose]
```

Argomenti:

Argomento	Default	Significato
<code>--db PATH</code>	<code>runts.db</code>	Percorso del file SQLite da arricchire.
<code>--verbose</code> , <code>-v</code>	<code>off</code>	Livello log <code>DEBUG</code> .

Output finale stampato su stdout (`geocoder.py:96-104`):

```
=====
REPORT GEOCODIFICA
=====
Già presenti      : <count>
Geocodificati     : <count>
Non trovati       : <count>
Saltati (no comune): <count>
=====
```

Funzione principale

`geocode_enti(conn: sqlite3.Connection) -> dict :`

1. Imposta `conn.row_factory = sqlite3.Row`.
2. Seleziona gli enti da geocodificare: `sql SELECT id_runts, sede_comune, sede_region FROM enti WHERE lat IS NULL OR lon IS NULL`
3. Per ogni riga:
4. Se `sede_comune` è vuoto/nulla → `skipped += 1`, log `DEBUG`, continue.
5. Chiama `_nominatim_query(comune, regione)`.

6. **Dorme 1 secondo** (`time.sleep(1)`) **dopo ogni richiesta**, indipendentemente dall'esito, per rispettare il rate limit di Nominatim.
7. Se ottiene (lat, lon) : esegue `UPDATE enti SET lat = ?, lon = ? WHERE id_runs = ?` e committa. Log INFO.
8. Se non trova coordinate: `not_found += 1` , log WARNING.
9. Ritorna `{"geocoded": N, "not_found": N, "skipped": N}` .

Il conteggio "Già presenti" viene letto **prima** della funzione (`geocoder.py:90`) con `SELECT COUNT(*) FROM enti WHERE lat IS NOT NULL AND lon IS NOT NULL` .

Query Nominatim (`_nominatim_query`)

```
_NOMINATIM_URL = "https://nominatim.openstreetmap.org/search"
_USER_AGENT = "runts-cai-geocoder/1.0 (https://github.com/piccioli/runts)"
```

Costruzione query:

1. Parti: `[comune.title()]` , se `regione` è valorizzata aggiunge `regione` , in coda `"Italia"` .
2. Join con `", "` → es. `"Pisa, Toscana, Italia"` .
3. Querystring: `q=<query>&format=json&limit=1` .
4. Richiesta `GET` con header `User-Agent` (obbligatorio per la policy Nominatim) e `timeout=10` .
5. Parsing JSON: se la lista è non vuota, estrae (`float(lat)` , `float(lon)`) dal primo risultato. Altrimenti ritorna `None` .

In caso di **qualsiasi eccezione** (rete, JSON malformato, timeout), logga warning e ritorna `None` — l'errore non blocca il batch.

Vincoli operativi

Vincolo	Valore	Sorgente
Rate limit Nominatim	1 req/s	Policy ufficiale OSM Nominatim.
User-Agent obbligatorio	sì	Policy: identificare l'applicazione.
Timeout per richiesta	10 s	Hardcoded in <code>_nominatim_query</code> .
Idempotenza	sì	Filtro <code>WHERE lat IS NULL OR lon IS NULL</code> salta gli enti già geocodificati.

Per ~226 enti senza coordinate, il batch dura circa **4 minuti** (rate limit + tempo di risposta).

Precisione e granularità

La query usa **comune + regione, non l'indirizzo completo**. Conseguenze:

- Le coordinate ottenute sono **a livello di centroide del comune**, non dell'indirizzo esatto.
- Buono per la mappa d'insieme a scala nazionale/regionale; meno preciso per la mappa di dettaglio del singolo ente (lo zoom 14 della pagina di dettaglio mostra comunque la città).
- Più affidabile rispetto a query con indirizzi RUNTS spesso incompleti o malformattati.

Decisione architetturale (vedi [openspec/changes/mappa-enti/design.md](#)): la precisione "civico esatto" è esplicitamente fuori scope.

Casi di fallimento

Caso	Comportamento
<code>sede_comune</code> NULL	Salto (<code>skipped</code>), log debug.
Nominatim non trova nulla	<code>not_found</code> , log warning, lat/lon restano NULL.
Errore di rete o timeout	Log warning con il messaggio dell'eccezione, lat/lon restano NULL.
JSON malformato	Idem: log warning, valore <code>None</code> .

In nessun caso il geocoder esce con codice di errore: termina sempre con il report finale e codice 0.

Esito tipico

Sul dataset attuale (~226 enti CAI), un'esecuzione completa geocodifica **>= 200 enti** correttamente. Le 20-26 mancate corrispondono solitamente a:

- Comuni con varianti ortografiche non riconosciute da Nominatim.
- Comuni con omonimi multipli risolti su un'area diversa (raro grazie all'aggiunta della regione).

Logging

Livello	Eventi
INFO	apertura DB, count "già presenti", ogni geocodifica riuscita (Geocodificato <id> (<comune>): <lat>, <lon>).
WARNING	errore Nominatim per query, ente non trovato.
DEBUG	ente senza comune saltato.

Formato: identico a quello dello scraper (`%(asctime)s %(levelname)-8s %(message)s , %H:%M:%S`).

Comportamenti non garantiti

- **Re-run dopo scraper:** se lo scraper viene rieseguito (e `lat / lon` non sono nel dict di `upsert`), le coordinate vengono azzerate dall' `INSERT OR REPLACE` . Il geocoder andrà rieseguito per ripopolarle.
- **Cambio di policy Nominatim:** l'endpoint pubblico ha limiti d'uso e potrebbe rifiutare richieste anomale; non ci sono fallback a server alternativi.
- **Mancanza di cache locale:** non c'è una tabella di lookup persistente (`comune` → `lat/lon`) condivisa tra esecuzioni; ogni geocodifica fa una nuova richiesta.

6. Web app (FastAPI)

Comportamento osservato di `web/app.py` e dei template Jinja2 in `web/templates/`.

Scopo

Servire una mini-applicazione di consultazione (HTML) per gli enti CAI registrati nel RUNTS. Non espone API JSON, non gestisce autenticazione, non permette scrittura.

Stack e bootstrap

Voce	Valore
Framework	FastAPI
ASGI server	Uvicorn (<code>uvicorn[standard]</code>)
Templating	Jinja2 via <code>fastapi.templating.Jinja2Templates</code>
Static	<code>StaticFiles</code> montato su <code>/static</code> (directory <code>web/static/</code> attualmente vuota)
DB driver	<code>sqlite3</code> stdlib in modalità URI <code>mode=ro</code>

Variabili e costanti chiave (`web/app.py`):

```
DB_PATH = os.environ.get("DB_PATH", "/app/runts.db")
PAGE_SIZE = 20
```

L'app viene avviata con:

```
uvicorn web.app:app --host 0.0.0.0 --port 8000
```

(comando definito nel `Dockerfile`.)

Route

GET /

Funzione: `enti_list(request, q, regione, sezione_registro, page)`.

Query string supportata:

Parametro	Tipo	Default	Effetto
<code>q</code>	<code>str?</code>	none	LIKE case-insensitive su denominazione (<code>%q%</code>).

Parametro	Tipo	Default	Effetto
regione	str?	none	Filtro esatto su sede_regione .
sezione_registro	str?	none	Filtro esatto su sezione_registro .
page	int	1	Pagina corrente, normalizzata a [1, total_pages] .

Comportamento

1. Se `runts.db` non esiste (`_db_exists()` False) → render di `list.html` con dataset vuoto e zero filtri disponibili. **Nessun 500.**
2. Apre il DB in sola lettura, costruisce dinamicamente la `WHERE` con i filtri attivi, eseguendo SEMPRE con parametri posizionali.
3. Conta il totale (`SELECT COUNT(*) FROM enti <where>`).
4. Calcola `total_pages = ceil(total / PAGE_SIZE)` (con minimo 1) e clampa `page` .
5. Estrae la pagina di risultati: `` `sql SELECT id_runts, denominazione, sede_comune, sede_regione, sezione_registro, lat, lon FROM enti <where> ORDER BY denominazione LIMIT ? OFFSET ?`  ``
6. Estrae le opzioni dei filtri (distinct sui campi `sede_regione` e `sezione_registro` , escludendo NULL, ordinati alfabeticamente).
7. Chiude la connessione (in `try/finally`).
8. Renderizza `list.html` con context: `enti` , `total` , `page` , `total_pages` , `q` , `regione` , `sezione_registro` , `regioni` , `sezioni` .

GET /ente/{id_runts}

Funzione: `ente_detail(request, id_runts, back)` .

Query string supportata:

Parametro	Tipo	Default	Effetto
back	str?	"/"	URL relativo a cui far tornare l'utente dal bottone "Torna alla lista".

Comportamento

1. Se `runts.db` non esiste → render di `404.html` con HTTP **404**.
2. `SELECT * FROM enti WHERE id_runts = ?` (param posizionale).

3. Se `row is None` → `404.html` con HTTP **404**.
4. Costruisce `fields = {k: row[k] for k in row.keys() if row[k] is not None and k not in ("id_runs", "raw_json", "updated_at")}` . Quindi:
5. I campi `NULL` **non finiscono** nel dict (la pagina di dettaglio non mostrerà la riga corrispondente).
6. `id_runs` , `raw_json` , `updated_at` sono esclusi dalla visualizzazione.
7. Estrae `lat / lon` dalla riga se presenti.
8. Render `detail.html` con context: `ente (sqlite3.Row originale)`, `fields` , `back` , `lat` , `lon` .

Template

`base.html`

Layout HTML5 in italiano (`<html lang="it">`):

- Bootstrap 5.3.3 via CDN:
`https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css` .
- Sfondo `#f8f9fa` .
- Navbar dark con brand `RUNTS` · Sezioni CAI linkato a `/` .
- Blocco `head` e blocco `content` espandibili dai template figli.

`list.html`

Eredita da `base.html` . Carica il CSS di Leaflet 1.9.4 via CDN solo in questa pagina (`{% block head %}`).

Sezioni della pagina

1. **Titolo:** `H1 "Enti iscritti al RUNTS"` .
2. **Form filtri** (`<form method="get" action="/">`): `input q` (testo libero), `select regione` (con `Tutte le regioni`), `select sezione_registro` (con `Tutte le sezioni`), pulsanti `Cerca` e `Reset` (link a `/`). Layout Bootstrap a 12 colonne (`row g-2`).
3. **Empty state:** se `total == 0` , alert info `"Nessun ente trovato."` . Nessun altro contenuto.
4. **Header risultati:** `"<total> enti trovati – pagina <page> di <total_pages>"` + button group `Lista / Mappa` (toggle JS, vedi sotto).
5. **Vista lista** (`#view-lista`):
6. Tabella Bootstrap `table-hover table-sm` con colonne: Denominazione (link a `/ente/<id_runs>?back=<back_url>`), Comune, Regione, Sezione.

MONTAGNA SERVIZI S.C.P.A.

Via Errico Petrella 19 - 20124 - Milano (MI)

<https://montagnaservizi.com>

PEC: montagnaserviziscpa@legalmail.it - email: info@montagnaservizi.com

C.F./P.IVA: 11790660960

7. Valori NULL renderizzati come `-`.
8. **Paginazione:** visibile solo se `total_pages > 1`. Mostra `<`, prime/ultime pagine, le 5 pagine intorno alla corrente, ellipsis (`...`) tra blocchi, `>`. Tutti i link preservano `q`, `regione`, `sezione_registro` nella query string.
9. **Vista mappa** (`#view-mappa`, `display:none` di default):
10. `<div id="map">` con altezza 520 px e bordo arrotondato.

Toggle Lista/Mappa

Implementato in JS inline:

- `showLista()` / `showMappa()` mostrano/nascondono i due div e aggiornano lo stile dei bottoni (`btn-primary` vs `btn-outline-primary`).
- `showMappa()` chiama `initMap()` (idempotente: ritorna subito se la mappa è già stata creata) e poi `map.invalidateSize()` per gestire correttamente il layout dopo il display.

Mappa Leaflet (lista)

- Tile layer: `https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png`, `maxZoom: 18`, con attribuzione OSM obbligatoria.
- Dati: array JS `ENTI_GEO`, generato dal template iterando su `enti` e includendo **solo** quelli con `lat` e `lon` valorizzati: `{id, nome, lat, lon}`.
- Marker per ogni ente con popup: `<strong>nome</strong><br><a href="/ente/<encoded_id>">Vedi dettaglio</a>`.
- Centra/zoom con `map.fitBounds(bounds, {padding: [30, 30]})`.
- Se `ENTI_GEO.length === 0`, mostra l'Italia (centroide `[42.5, 12.5]`, zoom 6) con popup "Nessun ente con coordinate disponibili".

*Nota: la mappa mostra **solo gli enti della pagina corrente** (limitata da `PAGE_SIZE=20`), non l'intero dataset filtrato. Questo è il comportamento attuale del template.*

`detail.html`

Eredita da `base.html`. Carica il CSS di Leaflet solo se `lat` e `lon` sono valorizzati.

Sezioni della pagina

1. **Link di ritorno:** bottone outline `← Torna alla lista` con `href="{{ back }}"`.
2. **Card principale:** header dark con denominazione, body con `<dl class="row">` che itera su una mappa fissa di label/campo:

<code>codice_fiscale</code>	→ Codice fiscale
<code>forma_giuridica</code>	→ Forma giuridica
<code>natura_giuridica</code>	→ Natura giuridica
<code>sezione_registro</code>	→ Sezione del registro

MONTAGNA SERVIZI S.C.P.A.

Via Errico Petrella 19 - 20124 - Milano (MI)

<https://montagnaservizi.com>

PEC: montagnaserviziscpa@legalmail.it - email: info@montagnaservizi.com

C.F./P.IVA: 11790660960

data_iscrizione	→ Data iscrizione
sede_stato	→ Stato
sede_indirizzo	→ Indirizzo
sede_civico	→ Civico
sede_comune	→ Comune
sede_provincia	→ Provincia
sede_region	→ Regione
sede_cap	→ CAP
rappresentante_legale	→ Rappresentante legale
pec	→ PEC
sito_web	→ Sito web
settori_attivita	→ Settori di attività

Una riga viene renderizzata solo se `fields.get(key)` è truthy. Tre rendering speciali:

- `pec` → `<a href="mailto:...">` .
- `sito_web` → `<a href="..." target="_blank" rel="noopener">` .
- Altri → testo semplice.

1. **Card footer** (condizionale su `ente.url_dettaglio`): bottone outline Vedi su RUNTS che apre l'URL ufficiale in nuova tab (`target="_blank" rel="noopener"`).

1. **Mappa Leaflet** (condizionale su `lat` and `lon`):

2. Card separata con header "Sede legale – mappa" e `<div id="map">` 380 px.
3. Tile OSM identico alla lista.
4. Centrata su `[lat, lon]` , zoom 14.
5. Singolo marker con popup popolato da `ente.denominazione | tojson` .

404.html

Pagina semplice in `text-center` :

- H1 "404" .
- `<p>Ente non trovato.</p>` .
- Bottone `← Torna alla lista` verso `/` .

Sicurezza

- **Read-only DB**: connessione SQLite via URI `mode=ro` (impossibile scrivere anche se ci fosse un bug).
- **Parametri posizionali ovunque**: nessuna concatenazione di stringhe in SQL.
- `tojson` e `urlencode` : usati nei template per i campi che entrano in JavaScript/URL, riducendo rischio di XSS o di URL malformati.
- Nessun cookie, nessuna sessione, nessun CSRF token (non necessario senza scrittura).

MONTAGNA SERVIZI S.C.P.A.

Via Errico Petrella 19 - 20124 - Milano (MI)

<https://montagnaservizi.com>

PEC: montagnaserviziscpa@legalmail.it - email: info@montagnaservizi.com

C.F./P.IVA: 11790660960

Comportamenti non garantiti

- **Mappa solo della pagina corrente:** l'utente che vuole vedere tutti gli enti filtrati deve passare in rassegna le pagine; non c'è una vista "tutti i marker contemporaneamente".
- **Nessuna cache HTTP:** ogni richiesta apre/chiude una connessione e rilegge il DB. Per il volume attuale è accettabile.
- **CDN-dependant:** senza internet o con CDN Bootstrap/Leaflet offline, l'interfaccia si degrada (HTML grezzo).
- **Nessun favicon:** la cartella `static/` è vuota.
- **Filtri esatti:** il filtro `regione` e `sezione_registro` non supporta valori multipli o ricerca parziale.

7. Deploy ed esecuzione

Configurazione di Dockerfile, docker-compose, esecuzione manuale di scraper/geocoder e gestione di `runts.db`.

Filosofia

Solo la **web app** viene containerizzata. Scraper e geocoder sono lanciati **dall'host** dall'operatore quando serve aggiornare i dati, perché:

- richiedono Playwright + browser headless, che pesano sull'immagine;
- vengono eseguiti raramente (manualmente, in modo batch);
- non c'è ragione operativa per averli sempre attivi.

Il file `runts.db` alla radice del progetto è il punto di sincronizzazione: lo scraper/geocoder vi scrivono dall'host, il container web lo monta in sola lettura.

Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY web/requirements.txt ./requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

COPY web/ ./web/

EXPOSE 8000

CMD ["uvicorn", "web.app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Punti chiave

- Base **python:3.12-slim**: image compatta, ufficiale.
- Si copia **solo** `web/requirements.txt` (non quello dello scraper): l'immagine non contiene Playwright né Chromium.
- Si copia **solo** la cartella `web/`: scraper non finisce nell'immagine.
- Nessun `runts.db` viene copiato dentro l'immagine: viene fornito a runtime via volume.
- `CMD` esegue uvicorn senza reload (servizio production-style).

docker-compose.yml

```
services:
  web:
```

```
build: .
ports:
  - "8000:8000"
volumes:
  - ./runts.db:/app/runts.db:ro
environment:
  - DB_PATH=/app/runts.db
restart: unless-stopped
```

Punti chiave

- **Bind mount in sola lettura** del DB host (`./runts.db:/app/runts.db:ro`): il container non può corrompere il file, e qualsiasi scrittura dall'host (scraper/geocoder) viene vista dal container al prossimo SELECT.
- Variabile `DB_PATH=/app/runts.db` letta da `web/app.py` (default coincidente).
- `restart: unless-stopped` : si rialza automaticamente in caso di crash, ma si ferma con `docker compose down` .
- Nessun altro servizio dichiarato: niente proxy reverse, niente DB separato.

Esecuzione tipica

Bootstrap del progetto

```
# 1. Crea ambiente virtuale per scraper/geocoder
python3 -m venv .venv
source .venv/bin/activate

# 2. Installa scraper deps (Playwright + pytest)
pip install -r scraper/requirements.txt
playwright install chromium

# 3. Prima esecuzione: popola il DB
python -m scraper.main --headless

# 4. Geocodifica
python -m scraper.geocoder --db runts.db

# 5. Avvia la web app
docker compose up -d
# → http://localhost:8000
```

Aggiornamento dati

```
# Lancia scraper (può girare con la web app attiva: SQLite WAL + ro mount)
source .venv/bin/activate
python -m scraper.main --headless

# Ri-geocodifica (lo scraper azzerà lat/lon dei record aggiornati)
python -m scraper.geocoder --db runts.db
```

La web app vede i nuovi dati al prossimo SELECT, senza riavvio.

Debug visivo dello scraper

```
python -m scraper.main --no-headless --delay 1000 --verbose
```

Browser visibile, pausa più lunga tra dettagli, log DEBUG.

Gestione del DB

Operazione	Modalità
Backup	Copia del file <code>runts.db</code> dall'host (è un singolo file).
Versionamento	<code>runts.db</code> , <code>runts.db-shm</code> e <code>runts.db-wal</code> sono in <code>.gitignore</code> : il DB non viene mai committato.
Reset	Eliminare il file <code>runts.db</code> e rilanciare lo scraper.
Migrazione schema	Automatica: <code>init_db()</code> esegue <code>CREATE TABLE IF NOT EXISTS + ALTER TABLE</code> idempotenti.

```
.gitignore corrente esclude: runts.db , runts.db-shm , runts.db-wal (file WAL),  
__pycache__/, *.pyc , .venv/ , venv/ , .claude/ .
```

Network e porte

- Container web esposto su `0.0.0.0:8000` .
- Nessuna comunicazione di rete interna tra container (servizio singolo).
- Nominatim e tile OSM sono chiamati direttamente dall'host (geocoder) o dal browser dell'utente (Leaflet); il container web non chiama nessun servizio esterno.

Risorse richieste

- **Web container:** ~50-80 MB RAM a regime, CPU trascurabile (workload I/O bound su SQLite locale).
- **Scraper (host):** dipende da Chromium headless (~300-500 MB RAM durante l'esecuzione).
- **Geocoder (host):** trascurabile (un singolo processo Python che dorme 1 s tra richieste).

Configurazioni note non implementate

- Nessuna pipeline CI/CD: build e deploy sono manuali.

- Nessun reverse proxy / TLS: in produzione si dovrebbe anteporre nginx/Caddy/Traefik.
- Nessun logging strutturato verso un aggregatore: i log restano in stdout del container.
- Nessun health check definito in compose (`healthcheck`) o nel Dockerfile.
- Nessun limite di risorse (`mem_limit` , `cpus`) nel compose.

Variabili d'ambiente complete

Variabile	Componente	Default	Note
<code>DB_PATH</code>	web app	<code>/app/runts.db</code>	Percorso del DB SQLite letto dal container.

Non esistono `.env` o secret manager; tutta la configurazione è in `docker-compose.yml` o negli argomenti CLI dello scraper.